

SKIjack: A High-Level Native Language for the *SKI* Calculus

N. E. Davis

September 18, 2026

Abstract

Over a homoiconic calculus the designer chooses where quotation happens. Over a calculus without self-quotation the choice is not available: no combinator can take another apart, so reification is metalevel by theorem, and a language’s staging boundary is fixed before its syntax is. We present SKIjack, a language for writing metacircular interpreters over the *SKI* calculus, whose feature set is designed off an existing artifact rather than designed: the 618-atom self-interpreter of Davis [2027], decomposed into its fuel loop, its dispatch, its spine walker and its atoms, with each part promoted to a surface construct. Declaring an object language generates that furniture, so authoring the semantics of a new symbol—the core operation performed by hand in Davis [2027]—becomes a two-line edit. Quotation is available at compile time only, and the checker refuses statically the positions at which a function would be asked to behave as data. A scry namespace is a language feature, with paths as a declared type and blocking sound under an append-only namespace.

1 Introduction

Over a homoiconic calculus (Lisp, Nock) the metacircular tradition gets quotation for free and the language designer chooses where evaluation stages live. Over a non-homoiconic calculus that choice is taken away: Proposition 3.1 says no term can quote a live term, so the compile-time/ runtime line is fixed before the language is designed, and the two operations the calculus cannot do for itself, quote and recognize, go to the compiler and the virtual machine respectively. Davis [2027] presented a metacircular evaluator for *SKI* which is written and evaluated in the same combinator system. This capability immediately suggests the construction of a battery of computational conveniences and abstractions within the same system; to wit, a language. We present SKIjack (pronounced “skyjack”), a closed-supercombinator macro-like language built in and targeting the non-homoiconic *SKI* calculus as its instruction set architecture. Every named thing compiles to a closed term and every name is resolved before anything runs, so the language has no runtime environment to address: bracket abstraction has already closed what a subject-oriented calculus would have to carry. SKIjack is evaluable on any *SKI* interpreter as a virtual machine.

The **whnfF** metacircular interpreter and **UQ** unquoter function presented in Davis [2027] are the starting point for our work. That paper discussed extensions to the *SKI* calculus built on higher layers of interpreters, introducing an **Err** error type, a **Maybe** partiality type, and discussed implications such as namespace value retrieval and other innovations. By authoring a new symbol at one site, the **whnfF** approach enables extension and modification of the combinator’s semantics, permitting it to behave more like a language than a mere combinator

system. This paper presents such a language, SKIjack, and three novel contributions for *SKI* combinator research: a two-level execution model with a kernel/ userspace split; and a namespace as a language feature: a resolver is a core parameter, paths are a declared type compared structurally, and the namespace literal compiles to a resolver.

2 Background

The calculus is the usual one. Terms are built from S , K and I by application alone, written left-associatively, with the rules $I x \rightarrow x$, $K x y \rightarrow x$ and $S f g x \rightarrow f x (g x)$. There are no variables and so no binding, no substitution and no environment; a surface language with binders reaches this calculus by bracket abstraction, which is the whole of the compiler’s back end [Turner, 1979b].

Data are Scott-encoded. A datum of an n -constructor type is a function of n continuations that applies the one belonging to its own constructor to its own fields, so a value *is* its own case analysis and there is no separate eliminator [Scott, 1962, Jansen, 2013, Böhm and Berarducci, 1985]. The order of the continuations is the order of the declaration, and it is part of the interface rather than a convention: a wrong continuation order is not a type error but silently different behaviour, which is why §9 treats case completeness and order as the first thing a checker owes an author.

The artifact this paper compiles to was built by Davis [2027]: a fueled normal-order interpreter for *SKI*, written in *SKI*, 618 atoms, with an unquoter **UQ** at 43 and structural equality on encoded terms at 240. Fuel is a Scott numeral and its semantics is exact — a budget of k permits k step-attempts, the last of which must be the no-redex check — so a run that performs s contractions needs $s + 1$. That paper reports the interpreter running a one-step object reduction in 340 host contractions, running the unquoter’s output in 91,556, and running *itself* in 504,930; it also proves the proposition this paper’s design is organized around, that no closed term can quote a live term. We reproduce those counts and do not re-derive them.

Four rulers get used on one object and are worth separating now, since a reader who confuses them will think the paper contradicts itself. The interpreter is 618 *atoms*; as a tree that is 1,235 *nodes*; sharing brings the graph to 348 *distinct nodes*; and printing it in the notation used here takes 2,109 *characters*. Atoms are the unit for every claim below, and the others appear only where they are named.

3 Lineage

SKIjack is a language entirely based on the *SKI* combinator calculus, and in particular on improving the expressiveness of *SKI* reasoning by the introduction of a standard machinery in the form of generated supercombinators. We survey the historical and technical lineage of pure combinator computation that informs SKIjack’s design. SKIjack is far from the first language or machine concept to lean into pure combinator computing. The present paper leans on five classic results from Corrado Böhm’s work:

1. *CUCH*. Böhm and Gross [1966] introduced the CUCH (CURry-CHurch), a surface for writing programs whose meaning is given by combinators. The *SKI* calculus (including SKIjack) operates under a more severe constraint: while CUCH’s designers were able to place its affordances such as quotation wherever they pleased, pure *SKI* calculus cannot quote expressions (Proposition 3.1 of Davis [2027]).

2. *Separability.* Böhm [1968] proved that two distinct $\beta\eta$ -normal forms can always be told apart by a context: applied to suitable arguments, one yields whatever result we care to name and the other does not. From the outside, distinct normal forms are separable; from the inside, no combinator may factorise another. The verification discipline used throughout this paper—apply a term to fresh markers and observe which one comes back—lives on the first side of that line, and so do the compiler’s quotation and the virtual machine’s jets; the calculus itself lives on the second. (Böhm’s theorem is stated for the λ -calculus and for $\beta\eta$ -distinct normal forms, and we take it for the asymmetry it indicates rather than as a completeness result for probing Scott-encoded data under weak reduction.)
3. *Term-algebra schema.* A type declaration generates its constructors and its case form by the schema Böhm and Berarducci [1985] gave for term algebras, in the Scott variant [Scott, 1962, Jansen, 2013] rather than the Church one: an interpreter’s step function wants case analysis in place of a fold which fixes the declaration order as the continuation order.
4. *First self-compiling compiler.* The first compiler written in its own language is recognized as Böhm [1954], his 1951 dissertation at ETH Zürich [Knuth and Pardo, 1980]. In the case of SKIjack, the compiler carries out quotation at compile time.
5. *Self-interpreter with a normal form.* Finally, Berarducci and Böhm [1993] gave a self-interpreter of the λ -calculus having a normal form.

Bracket abstraction permits the procedural compilation of a high-level applicative language to combinators [Turner, 1979a]; there is a related abstraction algorithm [Turner, 1979b]. Hughes [1982] introduced the notion of a supercombinator, the closed abstraction compiled as one unit. This program effloresced in the “SKI Machine” (SKIM), a graph-reduction machine built on the “Turner set” of combinators, namely, S , K , I , B , C , Y , and P [Johnsson, 1984, Clarke et al., 1980] and its successor [Stoye et al., 1984, Stoye, 1985]. (A standard account of the whole pipeline has been given by Peyton Jones [1987].) The SKIM tradition operated with a fixed set of recognized combinators executed natively in microcode, an accelerated process comparable to the “jet” approach we will adopt in this work. Much of this work was concerned with the cost of any particular abstraction, in combinators per source symbol [Noshita, 1985, Joy et al., 1985], and we report atom counts in line with that tradition.

In the preceding work, the interpreter lived either in a machine (Turner) or in microcode (SKIM), with the reduction engine as an external given. The object introduced by Davis [2027] is a 618-atom self-interpreter which exists as a term within the language it interprets. Previous supercombinator work has had no reason to write such an artifact; its self-application question was the compiler bootstrapping in its own source language, not the reducer reducing itself, and so the tower, the per-level cost, and the encoding boundary never arose there.

However, pursuant to the introduction of a self-interpreter as a term within the language, new considerations arise. Staging becomes a design problem: Turner’s SASL could not quote either, but because no SASL program was trying to run another SASL program under a fuel budget, there was no obligation to address the lack. Declaring an object language becomes a language feature, since an interpreter’s constructors, case form, spine walker, rebuilder and fuel loop can be generated from the declaration rather than written. Finally, the native compiler owes its output a strict discipline: expansion is bijective, terms lift back to named source, and the dictionary is published and keyed by structural hash. This discipline becomes crucial because the emitted term is simultaneously the object of study and the key a VM matches on, whereas nobody expected to read the output of a SASL compiler.

This last merits an aside. Our VM is capable of identifying supercombinators to arbitrary

size and reducing them instantly against a table of known outcomes in native code (what we call a “jet”, short for “jet-accelerated code”). Mechanically, such a procedure corresponds to Turner’s extended combinator set and SKIM’s microcode. However, our theorem does not licence arbitrary native code replacement; the interpreter jet performs a structural recognition that no term can perform for itself and thus is not an optimization of an operation the language has. The operational consequence of this prohibits a jet from quoting on behalf of a running term. The VM is addressed briefly in §11.

4 The interpreter

The **whnfF** function forms the centerpiece of the interpreter, responsible for reducing terms to weak head normal form. It orchestrates the evaluation strategy for the language, ensuring that terms are reduced correctly according to the operational semantics. It also suggests the language features that are necessary to support such an evaluation strategy, which will emerge in SKIjack as primitives. There is also an unquoter function **UQ**, responsible for converting weak head normal forms back into their original syntactic representation. This is necessary for maintaining the correspondence between the evaluated terms and their original syntactic forms, ensuring that the interpreter’s output can be related back to the source code. In Davis [2027], we simply named the features by their operational significance as a shorthand; here they become language elements in their own right.

Table 1 gives some typed forms with their defining equations and with closed $\{S, K, I\}$ terms emitted by the expander (i.e., compiled output). Three of the six reduce to the same term, and two more are a second same term. This results because a Scott datum is its own case analysis, so the first constructor of any two-constructor type discards the remaining continuation and is K , whatever the type was called, and a one-field second constructor applies one continuation to one field and is the same eight atoms whether the field is a numeral or a term. At the level of terms the type distinctions are erased. SKIjack maintains the type discipline at compile time (§9). A checker may reject a program that offers a numeral where a list is wanted, but nothing downstream of the expander could tell the two apart, because downstream of the expander they are one object. (It is also why the dictionary that names these expansions is keyed by the hash of a term and not by a name.)

Table 1: Some significant forms, their definitions, and the terms they compile to. The first constructor of each pair is the same single atom and the one-field second constructors are the same eight; only **cons**, which carries two fields, is distinct.

form	definition	expansion	atoms
nil	$\lambda n. \lambda c. n$	K	1
cons $h\ t$	$\lambda n. \lambda c. c\ h\ t$	$S\ (K\ (S\ (K\ K)))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K)))\ (K\ K))$	22
nothing	$\lambda n. \lambda j. n$	K	1
just x	$\lambda n. \lambda j. j\ x$	$S\ (K\ K)\ (S\ (K\ (S\ I))\ K)$	8
zero	$\lambda z. \lambda s. z$	K	1
suc n	$\lambda z. \lambda s. s\ n$	$S\ (K\ K)\ (S\ (K\ (S\ I))\ K)$	8

The metacircular evaluator uses these object terms to represent the basic constructs of the language within itself. Next, **whnfF** itself is composed of 618 atoms built from a few pieces. Table 2 inventories **whnfF**’s parts, smallest first, with the term each part compiles to. By

reading the lower block from the bottom upwards, the interpreter assembles itself out of the upper one.

- | | |
|---|--|
| 1. hd , the head extractor; | 14. stepK2 , the second K step equation; |
| 2. tl , the tail extractor; | 15. stepI , the I step equation; |
| 3. resI , the I result equation; | 16. stepK1 , the first K step equation; |
| 4. resK , the K result equation; | 17. stepK , the K step equation; |
| 5. Y , the fixed-point combinator; | 18. sp , the spine walker; |
| 6. resS , the S result equation; | 19. stepS3 , the third S step equation; |
| 7. pair , the pair constructor; | 20. stepS2 , the second S step equation; |
| 8. App , the application constructor; | 21. stepS1 , the first S step equation; |
| 9. UQ , the unquoter; | 22. stepS , the S step equation; |
| 10. rb1 , the first rebuilder equation; | 23. step , the step walker from the spine to the head leaf; |
| 11. spApp , the spine application handler; | 24. loop1 , the first loop equation; and |
| 12. rb , the rebuilder; | 25. whnfF , the whole interpreter. |
| 13. stepI1 , the first I step equation; | |

Note that the expansions of **rb**, **UQ**, **sp**, and **whnfF** begin with the fourteen atoms of Y . The fixpoint tie is a prefix of the emitted term, and which supercombinators recurse can be read off the object itself. This is perhaps the one piece of the surface’s structure that survives bracket abstraction in a form a reader can observe for himself.

5 The language

The **whnfF** function of Davis [2027] introduced a number of independent operational terms within its full expression, and these suggest language features such as its loop, its comparison and dispatch, and its atoms (**suc**, **zero**, **nil**, **just**, etc.). These features were composed into larger machine terms, which are similar to the supercombinators of Turner [1979a].

SKIjack’s features are written as a set of object terms in the metacircular evaluator; in essence, it is a macro language with a gently typed surface discipline. They name and highlight the components of **whnfF** and other recurrent supercombinators, augmented with affordances to make the language expressive and composable.

A language description of SKIjack is provided in Appendix A.

5.1 Design principles

Since quotation of live terms is not supported, the language relies on the macro-like composition of its object terms rather than runtime evaluation. A hard line between runtime quotation and compile-time quotation is forced by Proposition 3.1 of Davis [2027], which follows from Theorem 3.2 of Jay and Given-Wilson [2011]: “No closed term Q over $\{S, K, I\}$ satisfies $Q M \rightarrow \langle M \rangle$ for every closed M .” Everything on the encoded side of that line is closed under the calculus, including running encoded terms, comparing them, and building cores. Everything that would turn a live value back into data is unavailable and must be staged via the virtual machine. One design problem is to make that line visible and comfortable. Another difference from traditional macro languages is that weak SKI reduction is call-by-name: arguments are never forced early, **K I Ω** is **I**, and no form needs thunking.

Table 2: Every named part of the **whnfF** interpreter, ordered by size. Above the rule, the closed $\{S, K, I\}$ term in full; below it, the same term named against the parts above by *lift*, which renames exact structural matches as a macro.

part	atoms	expansion
hd	4	$S\ I\ (K\ K)$
tl	5	$S\ I\ (K\ (K\ I))$
resI	11	$S\ (K\ K)\ (S\ (K\ K)\ (S\ (K\ (S\ I))\ K))$
resK	13	$S\ (K\ K)\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ I))\ K))$
Y	14	$S\ (K\ (S\ I\ I))\ (S\ (S\ (K\ S)\ K)\ (K\ (S\ I\ I)))$
resS	15	$S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ I))\ K))$
pair	17	$S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (K\ K)$
App	32	$S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (K\ K)))$
UQ	43	$S\ (K\ (S\ I\ I))\ (S\ (S\ (K\ S)\ K)\ (K\ (S\ I\ I))\ (S\ (K\ (S\ (S\ (S\ I\ (K\ S))\ (K\ K))\ (K\ I))))\ (S\ (K\ K)\ (S\ (S\ (K\ S)\ (S\ (K\ (S\ I\ (K\ S))\ (S\ (K\ K))))\ K)))$
rb1	44	$S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ K)))\ (K\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (K\ K))))))$
spApp	48	$S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ (K\ S)))\ (S\ (K\ K))))\ (K\ (S\ (K\ K)\ (S\ (K\ (S\ (S\ (K\ (S\ (K\ K)))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (K\ K))))\ K)))$
rb	74	$S\ (K\ (S\ I\ I))\ (S\ (S\ (K\ S)\ K)\ (K\ (S\ I\ I))\ (S\ (K\ (S\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (S\ (K\ (S\ (K\ K))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ (K\ K))\ (S\ (K\ (S\ (K\ K))\ (S\ (K\ (S\ (K\ K))\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (K\ K))))))\ K)))$
stepI1	86	$S\ (K\ (S\ (K\ just)))\ rb$
stepK2	89	$S\ (K\ K)\ stepI1$
stepI	92	$S\ hd\ (K\ stepI1)$
stepK1	99	$S\ (K\ (S\ hd))\ (S\ (K\ K)\ stepK2)$
stepK	105	$S\ hd\ (K\ stepK1)$
sp	126	$Y\ (S\ (K\ (S\ (S\ (K\ S)\ (S\ (S\ (K\ S)\ (S\ (S\ (K\ S)\ (S\ (S\ (K\ S)\ K)\ (K\ resS)))\ (K\ resK)))\ (K\ resI))))\ (S\ (K\ K)\ spApp))$
stepS3	207	$S\ (K\ (S\ (K\ (S\ (K\ (S\ (K\ Just))))))\ (S\ (K\ (S\ (K\ (S\ (K\ rb))))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ (K\ App)))\ App))))\ (K\ App)))$
stepS2	221	$S\ (K\ (S\ (K\ (S\ hd))))\ (S\ (K\ (S\ (K\ K)))\ stepS3)$
stepS1	231	$S\ (K\ (S\ hd))\ (S\ (K\ K)\ stepS2)$
stepS	237	$S\ hd\ (K\ stepS1)$
step	569	$S\ (S\ (S\ (S\ sp\ (K\ K))\ (K\ stepS))\ (K\ stepK))\ (K\ stepI)$
loop1	588	$S\ (K\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ step\ Just))))\ K$
whnfF	618	$Y\ (S\ (K\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ hd))))\ (S\ (K\ K)\ loop1))$

The basic components of SKIjack are thus: a macro layer; Scott encoding; recursion through Y ; interpreters, the unquoter, and equality on encoded terms like paths; the behavioral verification discipline; and the existence of a runtime which can jet the compiled code for more efficient execution.

SKIjack supplies one grammar with two lexicons (ASCII-only and Unicode), with a bijective token table checked at import, and the round-trip and lift laws as regression tests. Given agent-driven development workflows, it is pragmatic to maintain both depending on the aesthetic and practical considerations of the development environment. Listing 1 and Listing 2 show the

Table 3: The object terms of *SKI* in the metacircular evaluator.

form	definition	expansion	atoms
nil	$\lambda n. \lambda c. n$	K	1
cons <i>h t</i>	$\lambda n. \lambda c. c h t$	S (K (S (K K))) (S (S (K S) (S (K K) (S (K S) (S (K (S I)) K)))) (K K))	22
nothing	$\lambda n. \lambda j. n$	K	1
just <i>x</i>	$\lambda n. \lambda j. j x$	S (K K) (S (K (S I)) K)	8
zero	$\lambda z. \lambda s. z$	K	1
suc <i>n</i>	$\lambda z. \lambda s. s n$	S (K K) (S (K (S I)) K)	8

same simple example. These are, by themselves, not a complete program: a namespace literal compares paths with **EQ5**, and **wfN** must be a core in scope, so a prelude is required. With those supplied, the program compiles to the results given in Table 4.

```
seg  ≡ Nat | Two | Three
path ≡ Nil | Cons seg path
resolve = ns{/nat/two ↦ <I>, /nat/three ↦ <K>}
answer  = wfN resolve ⊢ <:/nat/three>10
```

Listing 1: A simple example of SKIjack code under the Unicode lexicon.

```
seg  === Nat | Two | Three
path === Nil | Cons seg path
resolve := ns{/nat/two => <I>, /nat/three => <K>}
answer := wfN resolve |- <?^/nat/three>@10
```

Listing 2: A simple example of SKIjack code under the ASCII lexicon.

Table 4: The artifact of Listing 2. **Cons** is the same twenty-two atoms as the **cons** of Table 1: a two-field constructor in second position is one term whatever the type it belongs to was called. **answer** has been lifted against the program’s dictionary.

form	atoms	expansion
Nat	4	S (K K) K
Two	2	K K
Three	3	K (K I)
Nil	1	K
Cons	22	S (K (S (K K))) (S (S (K S) (S (K K) (S (K S) (S (K (S I)) K)))) (K K))
resolve	5,375	omitted for brevity
answer	9,007	wfN resolve (suc (suc (suc (suc (suc (suc (suc (suc (suc (suc K))))))))))

A three-segment path costs 2,485 atoms once quoted because at level 1 every constructor application is itself encoded, and **resolve** costs 5,375 because the namespace literal expands to a chain that compares the incoming path against each key with **EQ5** at 240 atoms, once per key, over full encodings of both. In reduced form, it gives **RValN K** in 28,661 contractions; the

sibling declaration that looks up `/nat/two` instead gives `RValN I` in 14,502, since the first key it tries is the one that matches. As can be seen, SKIjack and its `whnfF` interpreter are far from concise, but they are deterministic.

5.2 Representation ABI

A handful of representation choices sit between the compiler and everything written in the language. We call this the representation ABI because it specifies the contract between separately produced compiled artifacts, not because there are bytes or linkers. A term emitted by this compiler must interoperate with terms built by hand, and with native code that recognizes them; the two would fail to compose if either disagreed about the order of continuations. In a Scott encoding that order is simultaneously the layout of a datum and the convention for calling it.

A cell `[a b]` is the Scott pair $\lambda c. cab$, whose only operation is to hand its components to a selector; the head is `p K` and the tail `p (K I)`. Axis addressing follows a binary tree numbering, with axis 1 being the whole, axis $2n$ the head of axis n and $2n + 1$ its tail, and an axis compiles to the corresponding chain of projections at one projection per bit. Axes only validly address cells; a term's application tree is code, which by Proposition 3.1 cannot be inspected by any term, and so the notation that reaches into data stops at that boundary.

A type declaration is an ordered constructor list, with the order specified in the ABI. The declaration generates each constructor as $\lambda fields. \lambda c_1 \dots c_n. c_i fields$ as well as the case form to apply a datum to n continuations in declaration order, per the schema of Böhm and Berarducci [1985] in its Scott rather than Church variant. Booleans are `K` for yes and `K I` for no, so a conditional is application to two continuations. Numerals are Scott numerals and serve as fuel; there is no native integer, and arithmetic is unary.

5.3 Kernel forms

The kernel contains about a dozen forms and is intended to remain fixed. Each form has exactly one expansion, given by a fold over the syntax tree: a name resolves in the compile-time table to a binder, a sibling equation, a constructor or a macro; juxtaposition denotes application; a cell is a pair; an axis pick is a projection chain; a lambda and an equation are bracket abstraction over their binders; a case applies a datum to one continuation per constructor in declaration order; a quotation is a Scott encoding computed at compile time; a macro is expanded at its use site; and the two evaluation forms hand a quotation to an interpreter. Everything else is composed of these primitives.

Because every rule is a fold, the expander has no semantics of its own: the same source gives the same closed term, atom for atom, deterministically and context-free. The surface grows in macros and in the library rather than in the compiler.

5.4 Supercombinators vs. macros

A supercombinator and a macro can look alike in source code but are not equivalent. An equation is lambda-lifted over its binders and over the sibling equations it calls, tied with `Y` where it recurses, and emitted once as a supercombinator. A supercombinator is a closed term forming a single node in the graph and shared by every use. In contrast, a macro has no runtime existence but is expanded into each use site, so it costs nothing to reach and is duplicated per use. Small things (derived combinators, booleans, projections) tend to be macros, while large things (an interpreter, equality on encoded data, the standard library) prefer to be supercombinators.

(Hughes proposed supercombinators as the alternative to compiling into a fixed combinator set, whose expansions blow up in size; here the fixed set is the non-negotiable instruction set, so each lifted definition is abstracted into $\{S, K, I\}$ after lifting.)

The structural consequence is more important than the cost. Compilation leaves the supercombinators intact and nested inside one another: in the compiled interpreter of §4, `sp` is a subterm of `step`, `step` of `whnfF`, `rb` of `stepS`, and 40 of the term’s 348 distinct nodes are entries of the program’s own dictionary. This makes a supercombinator liftable back to its name, and lets a runtime recognize it (§11); a large open term offers neither improvement. Macros are hygienic by default, since a capture is invisible in source and shows up only as a body silently reading the wrong binding; a marked capturing form is reserved for the cases that want it.

5.5 The standard subject

A language of this shape wants a library, and the library wants to arrive as data rather than as a compiler built-in. We call this a subject in the tradition of subject-oriented programming. The virtual machine applies a loaded program to one argument, the *subject*, which carries the library at fixed axes—the primitives and Y , pairs and projections, Scott lists and numerals, equality on encoded terms, the unquoter, and the interpreters themselves as ordinary supercombinators. (A subject is not the same as an environment: names are resolved in a compile-time table and are gone before anything runs, and no compiled program applies itself to the subject to find its own names.)

5.6 The quotation line

So far, SKIjack is a fairly ordinary supercombinator language. The line that separates it from SASL or SKIM is where a term may be turned into data given that quotation is only available at compile time. The form `<t>` emits the Scott encoding of `t` against the enclosing program’s object type, computed by the expander; inside the brackets a name is a constructor of that type if it is one and is otherwise inlined and encoded in place. There is no form, and *there can be no form*, that takes a live value and yields its code. Even if the base representation is always $\{S, K, I\}$, SKIjack and the `whnfF` interpreter are not truly homoiconic, as code can never cross back over into data.

SKIjack thus makes this refusal explicit and visible rather than silent through its modest type system. Equality is offered on encoded data and refused on functions; a namespace key is a value of a declared path type and not an arbitrary term; a quotation’s argument is source and not an expression to be evaluated first. Each of those would require a function would have to behave as data, so the checker rejects a program that puts one there before the expander is reached (§9).

6 Execution model

A declaration runs at one of two levels:

- *Level 0*: the compiled term is handed to a reducer as it stands. This is ordinary evaluation: weak reduction is call-by-name, arguments are never forced early, and no form needs thinking. Nothing blocks and nothing can be observed partway; a term either reaches weak head normal form or it does not.

- *Level 1*: a quotation is handed to an interpreter together with a fuel budget, written `<t>@n`, or `I | - <t>@n` to name an interpreter other than the default. This compiles to an application: the interpreter, its parameters, the Scott numeral, and the quoted program, as the lifted form in §5 shows. Fuel must be explicit because a budget of k permits k step-attempts, of which the last must be the no-redex check, so a computation of s contractions needs $s + 1$. The budget cannot be inferred by the compiler, since a term’s running time is its running time and convergence is only semi-decidable. Elided fuel, written `@[]`, is therefore a *runtime policy* rather than a compile-time value: the budget doubles from a small start until the program produces something other than a timeout or a configured ceiling is reached, and the ceiling is reported as a timeout rather than hidden. (The one case a compiler could fill in is a term it can prove affine, and that case is not interesting enough to special-case.)

The levels differ in what is observable. At level 1, a program’s steps are counted, its divergence is bounded, its errors are values, and its scry operation can block. All of these are available because the interpreter is itself an ordinary term the program may name. Virtualization is not cheap, in any case: Davis [2027] measured the base interpreter at 340 host contractions for a single object step.

7 Interpreters as declarations

A central move of Davis [2027] was to author the semantics of a new symbol at one site of an interpreter. It performed that move by hand, in a definitional layer with no surface language above it. With SKIjack it becomes a declaration. Before extending the interpreter, though, we reproduce it: Listing 3 defines `whnfF` in SKIjack and it compiles atom for atom to the term printed in Appendix B of Davis [2027]; this compiler did not exist when that term was built, arriving at the same 618 atoms.

A type declaration names an object language, and from it the compiler generates what an interpreter over that language needs: the constructors and the case form, the spine walker that decomposes an application into a head and an argument list, the rebuilder that puts one back together, a default step equation for each leaf of the object type, and the fuel loop, whose shape is read off the outcome and result types. The timeout constructor is the last terminal of the result type, which for `maybe` is `Nothing`.

The original paper’s observability pair here reduces to two interpreters over a five-constructor object language, differing at the single equation that handles the new leaf, as two lines:

```
wf5Abs := { stepErr acc = Errd }
wf50mg := { stepErr acc = omega }
```

The first absorbs the new symbol into an error outcome; the second diverges on it. They compile to 768 and 771 atoms, again matching the artifacts of Davis [2027] atom for atom and they reproduce that paper’s observability table. In SKIjack, everything else about them, some seven hundred atoms of walker, rebuilder, step equations and loop, is generated from the three type declarations above them. (Incidentally, this shows that for *SKI* constructions, the simple SKIjack compiler is on par with hand construction of *SKI* expressions.)

```

term ≡ S | K | I | App term term
maybe ≡ Nothing | Just term
whnfF ≡ {
  step m = sp m nil stepS stepK stepI
  loop1 f m n2 = step m (Just m) (f n2)
  loop n m = n Nothing (loop1 loop m)
}
resS acc c0 c1 c2 = c0 acc
resK acc c0 c1 c2 = c1 acc
resI acc c0 c1 c2 = c2 acc
spApp f acc t u = f t (cons u acc)
sp m acc = m (resS acc) (resK acc) (resI acc) (spApp sp acc)
rb1 f h x xs = f (App h x) xs
rb h args = args h (rb1 rb h)
stepI1 x rest = Just (rb x rest)
stepI args = args Nothing stepI1
stepK2 x y rest = Just (rb x rest)
stepK1 x r = r Nothing (stepK2 x)
stepK args = args Nothing stepK1
stepS3 x y z rest = Just (rb (App (App x z) (App y z)) rest)
stepS2 x y r2 = r2 Nothing (stepS3 x y)
stepS1 x r = r Nothing (stepS2 x)
stepS args = args Nothing stepS1

```

Listing 3: The `whnfF` function defined in SKIjack. Pleasingly, this program produces the identical *SKI* artifact as Appendix B of Davis [2027] despite SKIjack not existing at the time of composition.

8 The namespace

The artifact of Davis [2027] accepts any term as a “s cry path”, or fixed address to value mapping. The language does not inherit that permissiveness, instead restricting valid s cry paths to the `path` type, a list of tagged segments, `path == Nil | Cons seg path`, and refusing on anything else. This is not merely tidiness: paths are compared, and comparison is only available on encoded data. An arbitrary term as a key would be a function in a position that requires a datum, which is forbidden (§9).

The resolver is a parameter of the interpreter core rather than a term closed over inside it. This makes a namespace a namespace: the core is written once, and supplying a different resolver does not recompile it. A namespace literal is sugar for building one. It expands to a chain that compares the incoming path against each key with `EQ5` over the encodings, answering with the oracle’s found-constructor at the first match and its not-yet-constructor when nothing matches, and the chain is an ordinary closed term like any other. Listing 2 and Table 4 show that arrangement as compiled and run.

A substantive contribution building on Davis [2027] is how the namespace handles a negative answer. The interpreter’s outcome and result types each carry an extra constructor that holds the path that blocked, and the driver, meeting one, records the fact and re-runs the program from the top under a resolver that now knows it. While re-running looks wasteful, it is the guarantor of the soundness. Because the namespace is append-only, a fact once learned is never

withdrawn, so a later run can only get at least as far as an earlier one; the blocked run is not resumed in the middle, where the intermediate state would have to be trusted, but discarded and replayed against more knowledge. We thus require the namespace to be fixed and referentially transparent; if a fact can change, replay proves nothing. Furthermore, fuel is not neutral across a resumption. A replayed run spends its budget again from the beginning, so the fuel a program needs is a function of how many times it blocked, and a budget that sufficed on the first attempt may not suffice on the third.

9 Boundary soundness

Proposition 3.1 is a statement about terms rather than programs. The language itself must enforce that the checker accepts only programs that the calculus can perform. In particular, no program may ask the calculus to perform an invalid operation; to wit, reification of code as data. There are four such positions:

1. An operand of equality.
2. The argument of a quotation.
3. A key of a namespace literal.
4. A scty path.

Each of these operations is thus required by the checker to be of a declared data type. The check is a pass over the syntax tree that runs before expansion and can only reject: it rewrites nothing. A well-typed occupant of each position is a datum and every other form is closed under the calculus. In addition, because checking cannot rewrite, the emitted term is the same whether the check ran or not, so the laws of §10 are unaffected by the presence of a type discipline and the calculus remains untyped when seen by the interpreter.

10 Implementation and evaluation

An initial compiler is provided as a Python package, **skijack**. Its front end consists two lexers over one token table and a single parser, so that the ASCII and Unicode surfaces are the same syntax; its middle is the checker of §9 and the generator of §7; its back end is macro expansion to a fixpoint, name resolution, lambda-lifting and bracket abstraction, each a total function on the tree.

Four laws are executable tests rather than remarks. Rendering is canonical, so $expand \circ render = lower$; lifting a term against the dictionary names exact structural matches and nothing else, so $lower \circ lift = id$; checking can reject but never rewrite, so expansion is the same with it on or off; and every dictionary entry is verified behaviourally, by applying the term to fresh markers and observing which one comes back, never by reading its syntax. The corpus is 17 programs written in both lexicons, and the suite is 705 tests.

The evaluation is a single claim, made by equality of terms. Six programs compile term-identically to artifacts Davis [2027] built by hand:

program	atoms	
whnfF	618	base interpreter; runs $\langle I \ K \rangle$ in T0's 340 contractions
wf5Abs	768	five-constructor interpreter, absorbing equation
wf50mg	771	the same, diverging equation
wfQ	950	blocking interpreter, oracle as a parameter
wfN	1,066	the same over a typed path namespace
EQ5	240	structural equality on encoded terms

No performance claim is made here. The reference host is an oracle and not an implementation: it reduces by substitution, counts every contraction, and is slow on purpose, because its counts are the claims. What a runtime would do with these terms is §11 and, in earnest, another paper.

11 The virtual machine

The reference reducer is an oracle, and a language that intends to be used wants an implementation. What that implementation must be is fixed, unusually, by a theorem rather than by taste. The two operations the calculus cannot perform for itself are quoting a live term and recognizing one's structure; the first belongs to the compiler, which has the syntax tree and quotes at compile time, and the second belongs to the runtime, which holds the term on its tape and may factorise it freely [Jay and Given-Wilson, 2011]. Matching a compiled subterm against a table of known terms and running native code for it — a jet — is therefore not an optimization of something the language has. It is the only place the operation exists.

The mechanism, we should say plainly, is old. Recognizing a fixed set of combinators and executing them natively is Turner's extended combinator set and SKIM's microcode [Turner, 1979a, Clarke et al., 1980, Stoye et al., 1984], and a graph reducer with sharing is the same vintage. What the theorem changes is the licence rather than the technique, and it yields exactly one rule: a jet may never quote on behalf of a running term, because the language guarantees to its users that nothing can.

Two properties of such a runtime bear on the claims made here, and both cut against convenience. Jets change contraction counts, so a jetted run cannot be compared with the counts of Davis [2027] at all. And a reducer that updates a redex in place shares work the reference reducer repeats, which lowers those counts even with no jets installed; the paper's numbers are the reference's substitution semantics, and an implementation that wishes to reproduce them must decline to share. Values are unaffected either way. A runtime is being built on these terms and is the subject of a separate paper; nothing in this one depends on it.

12 Limitations

SKIjack has evaluation of encoded terms, including terms built at run time: that is what an interpreter applied to a datum is, and the tower of Davis [2027] is nothing else. What it lacks is reification of a live value. The missing operation is *reify*, not *eval*, and the distinction is the subject of this paper rather than a quibble about names.

The language is not self-hosting. While self-compilation would need a parser mapping text to the internal representation of SKIjack terms, the real reason lies in the interpreter tower. The Futamura projections inside the calculus mean that an expander that is a term could be run

under an interpreter and specialized, and the tower would gain a level with new content rather than another copy of the same interpreter. This is not prohibitive but has not been undertaken. The operational overhead does not seem to merit the additional effort or reveal new premises for *SKI* as a tool.

The type discipline stops at the stage described in §9. The standard subject of §5 is specified but unbuilt, so library names are inlined and level-1 programs do not share a quoted library. Qualified names parse and do not yet resolve.

While the language is Turing-complete, certain functions are transferred to the virtual machine host, such as scaffolding the initial evaluation, preserving the state and memory arena, and providing the supercombinator jet acceleration table.

13 Conclusion

The line of research leading to the original SKI Machine focused on the development of combinatory logic as a foundation for implementing functional programming languages efficiently. These techniques formed the basis for the first lazy functional language implementations, providing a practical, variable-free model for executing functional programs without the overhead of variable management. The exploration of supercombinator techniques by SKIjack and its virtual machine(s) may spur further reflection and development in the field of efficient functional language implementation, as well as reconceptualization of the role of combinatory logic in the foundations of computer science.

References

- Alessandro Berarducci and Corrado Böhm. A self-interpreter of lambda calculus having a normal form. In *Computer Science Logic (CSL '92)*, volume 702 of *Lecture Notes in Computer Science*, pages 85–99. Springer, 1993.
- Corrado Böhm. Calcolatrici digitali: du déchiffre de formules logico-mathématiques par la machine même dans la conception du programme. *Annali di Matematica Pura ed Applicata*, 37:175–217, 1954. Series 4. Originally the author’s doctoral dissertation, ETH Zürich, 1951.
- Corrado Böhm. Alcune proprietà delle forme β - η -normali nel λ -K-calcolo. Technical Report 696, Istituto per le Applicazioni del Calcolo, Rome, 1968.
- Corrado Böhm and Alessandro Berarducci. Automatic synthesis of typed λ -programs on term algebras. *Theoretical Computer Science*, 39:135–154, 1985.
- Corrado Böhm and Wolf Gross. Introduction to the CUCH. In Eduardo R. Caianiello, editor, *Automata Theory*, pages 35–65. Academic Press, New York, 1966.
- T. J. W. Clarke, P. J. S. Gladstone, C. D. MacLean, and A. C. Norman. SKIM — the S, K, I reduction machine. In *Proceedings of the 1980 ACM Conference on LISP and Functional Programming*. ACM, 1980.
- N. E. Davis. A self-interpreter for *SKI* written in *SKI*: Extensional semantics for new symbols. *Journal of Functional Programming*, 2027. In review.

- R. John M. Hughes. Super-combinators: A new implementation method for applicative languages. In *Proceedings of the 1982 ACM Symposium on LISP and Functional Programming*. ACM, 1982.
- Jan Martin Jansen. Programming in the λ -calculus: From Church to Scott and back. In *The Beauty of Functional Code*, volume 8106 of *Lecture Notes in Computer Science*, pages 168–180. Springer, 2013.
- Barry Jay and Thomas Given-Wilson. A combinatory account of internal structure. *Journal of Symbolic Logic*, 76(3):807–826, 2011.
- Thomas Johnsson. Efficient compilation of lazy evaluation. In *Proceedings of the SIGPLAN Symposium on Compiler Construction*. ACM, 1984.
- Mike S. Joy, V. J. Rayward-Smith, and F. Warren Burton. Efficient combinator code. *Computer Languages*, 10(3–4), 1985.
- Donald E. Knuth and Luis Trabb Pardo. The early development of programming languages. In N. Metropolis, J. Howlett, and Gian-Carlo Rota, editors, *A History of Computing in the Twentieth Century*, pages 197–273. Academic Press, New York, 1980.
- Kohei Noshita. Translation of Turner combinators in $O(n \log n)$ space. *Information Processing Letters*, 20(2), 1985.
- Simon L. Peyton Jones. *The Implementation of Functional Programming Languages*. Prentice Hall, 1987.
- Dana S. Scott. A system of functional abstraction. Lectures delivered at the University of California, Berkeley, 1962/63; unpublished manuscript, Stanford University, 1963, 1962.
- William R. Stoye. *The Implementation of Functional Languages Using Custom Hardware*. PhD thesis, University of Cambridge Computer Laboratory, 1985. Technical Report 81.
- William R. Stoye, T. J. W. Clarke, and A. C. Norman. Some practical methods for rapid combinator reduction. In *Proceedings of the 1984 ACM Symposium on LISP and Functional Programming*. ACM, 1984.
- David A. Turner. A new implementation technique for applicative languages. *Software: Practice and Experience*, 9(1):31–49, 1979a.
- David A. Turner. Another algorithm for bracket abstraction. *The Journal of Symbolic Logic*, 44(2):267–270, 1979b.

A Language description

SKIjack recognizes one grammar but supports two lexicons. (The Unicode lexicon typically replaces ASCII digraphs.) Table 5 lists the tokens and their corresponding representations. The grammar, with Unicode spellings, is given in Listing 4; newlines terminate a declaration and an equation, and are invisible inside brackets, a case block and a namespace literal.

A quotation may nest inside another quotation. Fuel is a subscript in the Unicode lexicon and `@n` in ASCII, with `@[]` (Unicode `⌈`) standing for the runtime policy of §6. Axes address cells and never names; qualified names parse and do not yet resolve.

Table 5: The token table. Below the rule are identifier glyphs: names rather than operators, and the only rows whose two spellings differ by a single character.

ASCII	Unicode	meaning
<code>:=*</code>	<code>⋮*</code>	macro definition
<code>:=!</code>	<code>⋮!</code>	capturing macro
<code>===</code>	<code>≡</code>	type declaration
<code>ns{</code>	<code>ns{</code>	namespace literal open
<code>?^</code>	<code>∴</code>	scry
<code> ></code>	<code>▷</code>	case
<code> -</code>	<code>⊢</code>	interpreter selection
<code>:=</code>	<code>⋮</code>	definition
<code>=></code>	<code>↦</code>	fact
<code>-></code>	<code>→</code>	signature arrow
<code>:</code>	<code>:</code>	signature colon
<code>=</code>	<code>=</code>	equation
<code> </code>	<code> </code>	type alternative
<code><</code>	<code><</code>	quotation open
<code>></code>	<code>></code>	quotation close
<code>\</code>	<code>λ</code>	lambda
<code>.</code>	<code>.</code>	name qualifier
<code>;</code>	<code>;</code>	branch separator
<code>,</code>	<code>,</code>	fact separator
<code>/</code>	<code>/</code>	path separator
<code>(</code>	<code>(</code>	group open
<code>)</code>	<code>)</code>	group close
<code>[</code>	<code>[</code>	cell open
<code>]</code>	<code>]</code>	cell close
<code>{</code>	<code>{</code>	block open
<code>}</code>	<code>}</code>	block close
<code>B</code>	<code>∘</code>	composition (B)
<code>C</code>	<code>□</code>	swap (C)
<code>W</code>	<code>⋈</code>	duplicate (W)
<code>Y</code>	<code>Y</code>	fixpoint (Y)
<code>EQ</code>	<code>≐</code>	equality on data

```

decl      := type-decl | sig | equation | macro | core | def | run
run       := NAME :='' [expr  $\vdash$ '] '<' expr '>' (SUB |  $()$ )'
type-decl := NAME  $\equiv$ ' ctor ('|' ctor)*
ctor      := CNAME TYPE*                                -- CNAME capitalized; fields are types
sig       := NAME ':' TYPE  $\rightarrow$ (' TYPE)*              -- optional; ignored by the expander
equation  := NAME NAME* '=' expr                        -- inside a core, or at top level
core      := NAME NAME* :='' '{' equation* '}'          -- parameters after the name
macro     := NAME NAME* :='*' expr | NAME NAME* :=!' expr
def       := NAME :='' expr
expr      := app
app       := atom+                                       -- left-associative
atom      := NAME | GLYPH | '(' expr ')'
           | '[' expr expr+ ']'                          -- cell
           | '<' expr '>'                                -- quotation: a datum
           | ':' atom                                     -- scry
           | NUMBER  $\sqsubseteq$ ' atom                       -- axis pick
           |  $\lambda$ ' NAME '.' expr
           | expr  $\triangleright$ ' '{' branch (';' branch)* '}'
           | 'ns{' fact (',' fact)* '}'
branch    := CNAME NAME* expr
fact      := path  $\rightarrow$ ' expr
path      := ('/' NAME)+

```

Listing 4: The grammar of the SKIjack language.

B The artifact

`whnfF` compiles to 618 atoms, identical to Appendix B of Davis [2027] (Listing 5).

[illegible]

Listing 5: The `whnfF` function fully expanded.